

MSXPi C BIOS – Developer Documentation

This document provides a complete technical overview of the MSXPi C BIOS source code, describing its architecture, available functions, and how each component participates in the MSX ↔ Raspberry Pi communication protocol. It is intended for developers extending MSXPi, integrating new commands, or maintaining the firmware.

1. Overview

The MSXPi BIOS is a low-level communication layer that allows MSX computers to exchange data with a Raspberry Pi using GPIO-based serial signalling. It implements:

- A reliable byte-transfer protocol
- Handshake and block-transfer mechanisms
- Error detection and recovery (checksums, ESC abort)
- Command-building utilities
- Fast screen output helpers

The BIOS is written in C using Fusion-C and custom MSX-DOS crt0, with careful attention to timing, reentrancy, and memory layout.

2. Printing Utilities

void Print(char* text)

Prints a null-terminated string to the screen.

- Basic text output function.

void PrintNumber(uint16_t value)

Prints a decimal number.

- Converts and outputs the number as ASCII digits.

void pprintf(char* text, uint16_t value)

Prints a string followed by a decimal number.

- Uses `Print()` and `PrintNumber()`.
- Useful for debugging numeric return codes.

void pprints(char* text, char* value)

Prints a string followed by another string.

- Convenience wrapper around `Print()`.

void FastPrint(char* text)

High-speed text output routine.

- First character printed via `PrintChar()` to set cursor position.
 - Remaining characters sent directly to VDP port 0x98.
 - Much faster than `Locate() + Print()`.
-

3. Low-Level GPIO Protocol

3.1 CHKPIRDY – Check Pi Ready State

`uint8_t CHKPIRDY(void)`

Core polling loop that determines the Pi's readiness state.

- Samples keyboard ESC every 256 passes (avoids BIOS keyboard overhead).
- Reads `CONTROL_PORT1` to detect protocol states:
 - `CHK_STATE_0`
 - `CHK_STATE_2`
- Returns `RC_ESCPRESSED` if ESC is detected.

This function is the timing-critical heart of the MSXPi protocol.

3.2 PIREADBYTE

`uint8_t PIREADBYTE(uint8_t* byte)`

Reads one byte from the Pi.

- Performs handshake via `CHKPIRDY()`.
- Supports legacy and new protocol versions.
- Returns `RC_SUCCESS` or error codes.

3.3 PIWRITEBYTE

`uint8_t PIWRITEBYTE(uint8_t byte)`

Writes one byte to the Pi.

- Uses `CHKPIRDY()` to ensure safe timing.
 - Returns `RC_SUCCESS` if Pi accepted the byte.
-

4. Block-Transfer Protocol

The MSXPi BIOS implements a robust block-based data transfer system with checksums and retransmission.

4.1 RECVDATA

```
uint8_t RECVDATA(uint8_t* dest, uint16_t* size, uint16_t*
maxbufsize)
```

Receives an entire multi-block payload from the Pi.

- Performs initial handshake.
- Receives blocks with:
 - header_rc
 - length
 - block index
 - payload
 - checksum
- Validates checksum and requests retransmission if needed.
- Returns total received size.

4.2 RECVDATA_ONEBLOCK

```
uint8_t RECVDATA_ONEBLOCK(uint8_t* dest, uint16_t* size, uint16_t
msx_blocksize)
```

Receives exactly one block.

- Used by `printstdout()` and other streaming operations.
- Enforces block index = 0.

4.3 SENDDATA2

```
uint8_t SENDDATA2(uint8_t* src, uint16_t size, uint16_t*
maxbufsize)
```

Sends data from MSX to Pi.

- Mirrors RECVDATA protocol.
 - Handles retransmission on checksum mismatch.
 - Performs status handshake after each good block.
-

5. Handshake Utilities

uint8_t PerformHandshake(uint16_t msx_blocksize)

Performs the initial READY/READY_ACK handshake.

- Sends MSX block size.
 - Used by both send and receive operations.
-

6. Command Line Processing

char* GetCmdLineParameters(void)

Reads MSX-DOS command tail from address 0x80.

- Trims leading spaces.
- Ensures null termination.

uint8_t SendCommandToMSXPi(const char* cmd, bool appendTail)

Builds and sends a command string to the Pi.

- Combines primary command and DOS tail.
 - Stores buffer at fixed address 0x8000 to avoid mapper conflicts.
 - Sends via SENDDATA2.
-

7. Error Handling

uint8_t parseConnError(const uint8_t rc)

Prints human-readable error messages for protocol return codes.

- Handles checksum errors, handshake failures, buffer overflow, file not found, etc.
-

8. Memory Utilities

uint16_t get_sp(void)

Returns current stack pointer.

uint16_t get_max_buffer_size(void)

Computes available buffer space based on heap top and stack.

uint8_t* get_buffer_ptr(void)

Returns pointer to buffer at heap top.

char* u16_to_ascii(uint16_t value, char* buf)

Converts a 16-bit integer to ASCII.

9. Streaming Output

uint8_t printstdout(uint8_t* buffer, uint16_t maxbufsize)

Receives text from Pi and prints it to the MSX screen.

- Uses RECVDATA_ONEBLOCK.
 - Prints each block as soon as it arrives.
-

10. Summary of Return Codes

Common return codes used throughout the BIOS:

- RC_SUCCESS – Operation completed
 - RC_READY – More blocks follow
 - RC_CHKSUM_ERR – Checksum mismatch
 - RC_CONNERR – Communication error
 - RC_HANDSHAKEERR – Handshake failure
 - RC_INVALIDCOMMAND – Command parsing error
 - RC_BUFOVFLW – Buffer overflow
 - RC_FILENOTFOUND – File not found on Pi
 - RC_ESCPRESSED – User aborted with ESC
-

11. Developer Notes

- Timing is critical: avoid calling BIOS routines inside interrupt hooks.
 - CHKPIRDY's ESC sampling is intentionally sparse for performance.
 - The command buffer is placed at 0x8000 to avoid mapper corruption.
 - All transfers use checksums for reliability.
-

12. Extending the BIOS

To add new MSXPi commands:

1. Build the command string using `SendCommandToMSXPi()`.
2. Implement the corresponding Python handler.
3. Use `SENDDATA2/RECVDATA` for binary transfers.
4. Use `printstdout` for text output.

This documentation should give you a clear and complete understanding of the MSXPi C BIOS architecture and how to use its functions effectively.